

AWS SAA-C03

Compute

EC2, Auto Scaling, ELB, ECS, EKS, Fargate, Lambda, Beanstalk

Solutions Architect Associate

Dispensa di Studio Approfondita

Amazon EC2: la spina dorsale del compute AWS

Amazon EC2 (Elastic Compute Cloud) è il servizio di calcolo fondamentale di AWS. Ti permette di lanciare server virtuali (chiamati istanze) con il sistema operativo, la CPU, la memoria e lo storage che preferisci. Capire EC2 in profondità è essenziale per l'esame SAA-C03 perché compare in quasi ogni scenario architetturale.

Famiglie di istanze: scegliere quella giusta

Le istanze EC2 sono organizzate in famiglie, ciascuna ottimizzata per un tipo di workload specifico. Il nome di un'istanza segue il formato: famiglia + generazione + attributi + dimensione (es. m5.xlarge).

Le istanze General Purpose (serie M e T) offrono un bilanciamento tra CPU, memoria e rete. La serie T (T3, T4g) ha un meccanismo di crediti burst: accumula crediti quando la CPU è sotto la baseline e li spende durante i picchi. Se i crediti si esauriscono, la performance cala. Questo le rende ideali per workload con utilizzo variabile come web server, ambienti di sviluppo e piccoli database. La serie M (M5, M6i, M7g) offre performance costanti senza burst, adatta per application server e backend.

Le istanze Compute Optimized (serie C) hanno il rapporto CPU/memoria più alto. Sono la scelta giusta per batch processing, encoding video, HPC, gaming server, machine learning inference e qualsiasi workload CPU-bound.

Le istanze Memory Optimized (serie R, X, z) hanno grandi quantità di RAM. La serie R è per database in-memory, cache distribuita e analytics real-time. La serie X ha ancora più memoria (fino a 4TB) per SAP HANA e database in-memory di grandi dimensioni.

Le istanze Storage Optimized (serie I, D, H) offrono alto throughput I/O verso storage locale (instance store). Ideali per database distribuiti come Cassandra, data warehouse e file system distribuiti come HDFS.

SUGGERIMENTO ESAME: Quando la domanda descrive un workload, identifica se è CPU-bound (C), memory-bound (R/X), storage-bound (I/D) o bilanciato (M/T). Questo ti guida verso la famiglia giusta.

Modelli di acquisto: ottimizzare i costi

Questo è uno degli argomenti più importanti dell'esame. AWS offre diversi modi per pagare le istanze EC2, e la scelta giusta può ridurre i costi fino al 90%.

On-Demand è il modello predefinito: paghi al secondo (Linux) o all'ora (Windows) senza impegno. È il più costoso ma il più flessibile. Usalo per workload imprevedibili, test, sviluppo o quando non puoi permetterti interruzioni.

Reserved Instances (RI) offrono sconti fino al 72% rispetto a On-Demand in cambio di un impegno di 1 o 3 anni. Le Standard RI sono legate a un tipo di istanza specifico e possono essere vendute sul Marketplace. Le Convertible RI

permettono di cambiare tipo di istanza durante il periodo ma offrono uno sconto inferiore (fino al 54%). Puoi scegliere di pagare tutto anticipato (All Upfront, sconto massimo), parzialmente anticipato (Partial Upfront) o nulla anticipato (No Upfront, sconto minimo).

Savings Plans sono piu' flessibili delle RI. Ti impegni su una spesa oraria (es. \$10/ora) per 1 o 3 anni. Compute Savings Plans si applicano a qualsiasi istanza EC2, Fargate e Lambda in qualsiasi regione. EC2 Instance Savings Plans sono legati a una famiglia di istanze in una regione specifica ma offrono uno sconto maggiore.

SUGGERIMENTO ESAME: Se la domanda parla di "workload stabile e prevedibile per 3 anni", la risposta e' Reserved Instances o Savings Plans. Se chiede "massima flessibilita' con risparmio", Compute Savings Plans.

Spot Instances offrono sconti fino al 90% ma possono essere interrotte da AWS con 2 minuti di preavviso quando la capacita' e' necessaria. Sono perfette per workload fault-tolerant: batch processing, analisi dati, CI/CD, rendering, training di modelli ML. Non usarle mai per database o applicazioni stateful che non possono gestire interruzioni.

Una strategia comune e' combinare On-Demand o RI per il carico base con Spot per i picchi. Questo si implementa con un ASG che usa un mix di istanze On-Demand e Spot (Mixed Instances Policy).

Dedicated Hosts sono server fisici interamente dedicati a te. Il caso d'uso principale e' il BYOL (Bring Your Own License) per software con licenze legate al server fisico, come Oracle, SQL Server o Windows Server con licenze per-socket o per-core.

Dedicated Instances girano su hardware dedicato ma non hai controllo su quale server fisico. Meno costose dei Dedicated Hosts, utili quando hai requisiti di compliance che impediscono la condivisione dell'hardware.

Placement Groups: controllare il posizionamento

I Placement Groups controllano come le istanze vengono distribuite sull'infrastruttura fisica.

Cluster posiziona tutte le istanze nella stessa rack (o rack adiacenti) nella stessa AZ. Offre la latenza di rete piu' bassa possibile e il throughput piu' alto (fino a 10 Gbps tra istanze). Usalo per HPC, applicazioni che richiedono comunicazione inter-nodo veloce. Il rischio e' che se la rack ha un problema, perdi tutte le istanze.

Spread distribuisce le istanze su hardware fisico diverso. Massimo 7 istanze per AZ per placement group. Ogni istanza e' su una rack separata con alimentazione e rete indipendenti. Usalo per applicazioni critiche dove vuoi massimizzare la disponibilita'.

Partition divide le istanze in partizioni logiche, ciascuna su rack separate. Fino a 7 partizioni per AZ, centinaia di istanze per partizione. Pensato per sistemi distribuiti come HDFS, HBase, Cassandra e Kafka che gestiscono la replica dei dati internamente.

SUGGERIMENTO ESAME: "Bassa latenza tra istanze" = Cluster. "Massima disponibilit  per poche istanze critiche" = Spread. "Database distribuito" = Partition.

Auto Scaling e Elastic Load Balancing

Auto Scaling Group (ASG)

Un Auto Scaling Group gestisce automaticamente il numero di istanze EC2 in base alla domanda. Definisci una capacita' minima (il minimo di istanze sempre attive), una capacita' massima (il limite superiore) e una capacita' desiderata (il target attuale).

Le scaling policy determinano quando e come scalare. Target Tracking e' la piu' semplice: specifichi un target (es. "mantieni la CPU media al 50%") e ASG aggiunge o rimuove istanze per raggiungerlo. Step Scaling definisce azioni diverse per soglie diverse (es. "aggiungi 2 istanze se CPU > 70%, aggiungi 4 se CPU > 90%"). Scheduled Scaling scala in base a un calendario (es. "aggiungi 10 istanze ogni lunedi' alle 8:00"). Predictive Scaling usa il machine learning per analizzare i pattern storici e scalare in anticipo.

Il cooldown period e' il tempo di attesa dopo un'azione di scaling prima che ne possa avvenire un'altra. Serve a evitare oscillazioni (scaling su e giu' continuo). Il default e' 300 secondi.

L'ASG esegue health check per verificare che le istanze siano funzionanti. Puo' usare gli health check EC2 (l'istanza risponde ai check di sistema?) o gli health check ELB (l'applicazione risponde correttamente?). Le istanze che falliscono vengono terminate e sostituite automaticamente.

Il Launch Template definisce la configurazione delle nuove istanze: AMI, tipo di istanza, security group, key pair, user data, IAM role. E' il successore del Launch Configuration e supporta funzionalita' aggiuntive come il versioning e il mix di tipi di istanza.

SUGGERIMENTO ESAME: Se la domanda chiede "come garantire che un'applicazione gestisca i picchi di traffico automaticamente", la risposta coinvolge quasi sempre ASG + ELB.

Elastic Load Balancer (ELB)

L'Application Load Balancer (ALB) opera al Layer 7 (HTTP/HTTPS) ed e' il tipo piu' usato. Supporta routing basato sul path (/api/* verso un target group, /images/* verso un altro), routing basato sull'hostname (api.example.com verso un target, www.example.com verso un altro), e routing basato su header, query string o IP sorgente. I target possono essere istanze EC2, indirizzi IP, funzioni Lambda o container. ALB supporta sticky sessions (tramite cookie), WebSocket, HTTP/2 e gRPC. Si integra con WAF per la protezione da attacchi web e con Cognito per l'autenticazione.

Il Network Load Balancer (NLB) opera al Layer 4 (TCP/UDP/TLS). E' progettato per performance estreme: gestisce milioni di richieste al secondo con latenza ultra-bassa (microsecondi). Ogni NLB ha un IP statico per AZ (puoi anche assegnare Elastic IP). Usalo quando hai bisogno di performance massime, protocolli non-HTTP (es. gaming, IoT, VoIP), o quando i client devono connettersi a un IP fisso.

Il Gateway Load Balancer (GWLB) opera al Layer 3 ed e' progettato per inserire appliance di rete (firewall, IDS/IPS, deep packet inspection) nel flusso del traffico in modo trasparente. Usa il protocollo GENEVE per incapsulare il traffico.

SUGGERIMENTO ESAME: "HTTP/HTTPS con routing avanzato" = ALB. "Performance massime, IP statico, TCP/UDP" = NLB. "Appliance di rete trasparenti" = GWLB.

Il Cross-Zone Load Balancing distribuisce il traffico uniformemente tra tutte le istanze registrate in tutte le AZ, indipendentemente dalla distribuzione del traffico tra le AZ. E' abilitato di default su ALB (gratuito) e disabilitato di default su NLB (a pagamento se abilitato).

Container: ECS, EKS e Fargate

Amazon ECS (Elastic Container Service) e' l'orchestratore di container nativo di AWS. Gestisce il ciclo di vita dei container Docker: deployment, scaling, networking, service discovery. Una Task Definition e' il blueprint che descrive uno o piu' container: quale immagine usare, quanta CPU e memoria allocare, quali porte esporre, quali volumi montare. Un Service mantiene un numero desiderato di task in esecuzione e li integra con un load balancer.

ECS puo' funzionare in due modalita': EC2 launch type, dove gestisci tu le istanze EC2 che ospitano i container, e Fargate launch type, dove AWS gestisce l'infrastruttura e tu ti preoccupi solo dei container. Fargate e' la scelta serverless: paghi per le risorse (vCPU e memoria) usate dal container, senza gestire server.

Amazon EKS (Elastic Kubernetes Service) e' Kubernetes gestito da AWS. Se la tua organizzazione usa gia' Kubernetes o ha bisogno di portabilita' multi-cloud, EKS e' la scelta giusta. Supporta sia EC2 che Fargate come worker nodes.

SUGGERIMENTO ESAME: Se la domanda menziona "Docker" o "container" senza specificare Kubernetes, pensa a ECS + Fargate. Se menziona "Kubernetes" o "portabilita'", pensa a EKS.

AWS Lambda e il paradigma serverless

AWS Lambda e' il servizio di compute serverless di AWS. Carichi il tuo codice (una funzione), definisci un trigger (un evento che la attiva) e Lambda si occupa di tutto il resto: provisioning dei server, scaling, patching, alta disponibilit . Paghi solo per il tempo di esecuzione effettivo, misurato in millisecondi.

Lambda ha alcuni limiti importanti da ricordare per l'esame: il timeout massimo e' 15 minuti (se il tuo processo richiede piu' tempo, Lambda non e' la scelta giusta), la memoria va da 128MB a 10GB (la CPU scala proporzionalmente alla memoria), e lo storage temporaneo in /tmp arriva fino a 10GB.

La concurrency e' il numero di esecuzioni simultanee. Il default e' 1000 per regione. La Reserved Concurrency riserva una quota per una funzione specifica (garantisce che abbia sempre capacita' ma limita anche il massimo). La Provisioned Concurrency mantiene un numero di istanze "calde" pronte a rispondere, eliminando il cold start (il ritardo della prima invocazione quando Lambda deve inizializzare un nuovo ambiente).

Lambda@Edge e CloudFront Functions permettono di eseguire codice ai margini della rete CDN di CloudFront. Lambda@Edge e' piu' potente (fino a 10 secondi, accesso a rete) ed e' utile per manipolazione di richieste/risposte, autenticazione, A/B testing. CloudFront Functions e' piu' leggera (sub-millisecondo, solo JavaScript) per manipolazioni semplici di header, URL rewriting, redirect.

SUGGERIMENTO ESAME: Se il workload e' event-driven, dura meno di 15 minuti e non richiede stato persistente, Lambda e' quasi sempre la risposta. Se la domanda menziona "cold start" come problema, la soluzione e' Provisioned Concurrency.

Elastic Beanstalk: PaaS su AWS

Elastic Beanstalk è la soluzione Platform-as-a-Service di AWS. Carichi il tuo codice e Beanstalk si occupa di creare e gestire l'infrastruttura: istanze EC2, Auto Scaling Group, Load Balancer, e opzionalmente un database RDS. Supporta Java, .NET, PHP, Node.js, Python, Ruby, Go e Docker.

Beanstalk offre diversi metodi di deployment: All at once (veloce ma con downtime), Rolling (aggiorna un batch alla volta), Rolling with additional batch (aggiunge istanze prima di aggiornare per mantenere la capacità), Immutable (crea un nuovo ASG con la nuova versione e poi fa swap), e Blue/Green (crea un ambiente completamente nuovo e usa Route 53 per switchare il traffico).

SUGGERIMENTO ESAME: Beanstalk è la risposta quando la domanda chiede "il modo più semplice per deployare un'applicazione web" o "ridurre la complessità operativa" senza andare completamente serverless.